When Specifications Drift: Measuring Semantic Preservation Across LLM Representations

Malhar Alpesh Patel[1] **Luiza Cristina Corpaci**[1,2]

[1]Apart Research [2]Vienna University of Technology

malharalpeshpatel@gmail.com luiza.corpaci@tuwien.ac.at

Abstract

Verification can establish that an implementation satisfies a specification, but not that a specification written or translated by a model preserves the initially intended meaning. We study this problem with 500 algebraic equations, each written in 18 representations, and ask language models to write each equation back in a standard notation (infix with explicit parentheses). A catalogue of approximately 22 million previously established implications from the Equational Theories Project tells us if the answer means the same as the original, permits more (is weaker), permits less (is stronger), or neither. Across 153,000 recorded calls, including repeats and three follow-up studies, the share of answers proven equivalent to the original (recovered) ranges from 73.7% when the law is spelled out in words to 8.5% for postfix notation. With look-alike variable names, 30.3% of responses are proven weaker, and 93.7% of those contain fewer distinct variables than the original. In a separate study that changes only the variable names, replacing ordinary letters with Latin/Cyrillic look-alikes lowers recovery from 64.4% to 3.1%. Explaining a notation with a worked example, or instructing the model not to transform the equation, recovers much of the loss for some models. These results show semantic failures in a controlled mathematical setting, where the failure frequency depends on how the task is presented. We argue that the evaluation of specification translation should state the limits of the checker explicitly and consider the direction of drift.

1 Introduction

When a language model rewrites a specification, it can change what the specification means. A verifier may then correctly prove that an implementation satisfies the rewritten specification, even if the implementation fails the original requirements. Verification asks if a program meets a given specification and validation asks if the specification says what was intended. Recent work on specification autoformalization shows the importance of this distinction and how validation matters in programming tasks [Agarwal et al., 2026].

Consider the law $x = x \diamond y$, where $\diamond$ is a binary operation, which must hold for all values of x and y. In one experiment, we used this law with the two variables renamed to the visually similar identifiers `I` and `l`. One model reconstructed the equation as $x = x \diamond x$, mistakenly treating the two variables as one. Every operation that satisfies the original law also satisfies the answer, while the operation "return the second argument" satisfies the answer, but violates the original. The answer is therefore *weaker*, as it permits implementations that the original rules out.

Specifications can be written in many ways: mathematical notation, proof-assistant syntax, code or natural language. We call each of these a *representation*. Although these may express the same requirements, a language model may interpret them differently. Earlier work shows that models are

NeurIPS 2026 Workshop on AI for Verifiable Coding.

sensitive to surface changes in prompt format [Sclar et al., 2024]. For verification, a relevant question is if a change of form makes the model wrong more often, or if different representations lead to different kinds of semantic errors.

We investigate this question using the Equational Theories Project (ETP), a collection of 4,694 small algebraic equations for which the logical relationship between almost every pair has been established by machine-checked proof [Bolan et al., 2025]. We select 500 equations and express each in 18 different representations of the same equation, including mathematical notation, programming languages, structured data, and English descriptions. We then ask nine language models to translate each representation back into a common mathematical notation. These results allow us to determine if the reconstructed equation is equivalent to the original, weaker (permitting more cases), stronger (permitting fewer cases), or incomparable (neither implies the other). We refer to this comparison procedure as our semantic oracle.

When the results cannot establish a relationship, we report that separately and make no assumption on the correctness of the reconstruction. Our main experiment consists of 81,000 model calls. Follow-up experiments examining the effects of variable naming, notation explanations, task wording, and repeated sampling bring the total to 153,000 calls.

Our contributions are threefold:

1. We introduce a controlled evaluation of how nine language models reconstruct 500 mathematical equations from 18 different representations. We distinguish reconstructions that preserve the original meaning from those that weaken, strengthen, or change it. We also distinguish semantic errors from outputs that cannot be evaluated.
2. We find that some representations are associated with characteristic error patterns, particularly the weakening of equations when variables have look-alike names.
3. We show that reconstruction depends on how notation is explained and how the task is phrased, alongside the notation itself. Explicit preservation instructions and more informative explanations can improve accuracy in some settings, although their effectiveness can vary across models and representations.

2 Related work

Language models can translate informal mathematics into formal statements and support subsequent theorem proving [Wu et al., 2022]. miniF2F provides formal mathematical statements across proof systems for theorem proving evaluation [Zheng et al., 2022]. These tasks differ from our controlled recovery task, which fixes the source mathematics and systematically changes its representation. Crucially, formal verification of a generated proof establishes the generated statement, not necessarily fidelity to its source. LeanEuclid addresses this problem directly: its Euclidean Equivalence Engine checks both implication directions between predicted and reference statements using SMT reasoning [Murphy et al., 2024]. We therefore do not claim that semantic evaluation or bidirectional checking is new. Our emphasis is a paired comparison across eighteen deterministic representations, with precomputed proven implications relations and analysis of error direction.

Verus-SpecGym evaluates programming-specification faithfulness using executable specifications, official tests, and adversarial cases [Agarwal et al., 2026]. FVEval scores LLM-generated SystemVerilog assertions by formal equivalence checking against reference assertions [Kang et al., 2024]. nl2postcond evaluates LLM-generated postconditions by checking if they accept correct implementations and reject buggy ones [Endres et al., 2024].O ur algebraic domain complements these practical settings with mapped identity pairs that can receive global semantic relations rather than only finite-test outcomes. Conversely, our domain omits program control flow, heap state, and tool-assisted repair, so it cannot establish corresponding effects on end-to-end verified code generation.

FormatSpread shows that prompt formatting can alter model performance and that format effects need not transfer between models [Sclar et al., 2024]. We study complete mathematical representations, including their notation notes, and measure weakening and strengthening alongside recovery. Trojan Source demonstrates security risks from discrepancies between displayed and interpreted source text [Boucher and Anderson, 2023]. Imperceptible character substitutions, including homoglyphs, also change the behavior of NLP models [Boucher et al., 2022]. Unicode mechanisms provide approaches

to detecting confusable identifiers [Unicode Consortium, 2025]. Our identifier experiment concerns models losing variable distinctions when recovering a universally quantified specification; it does not reproduce a compiler attack or test a defense.

3 Measuring semantic preservation

3.1 Domain and directional oracle

A magma is a set equipped with a binary operation, written $\diamond$, with no assumed properties such as associativity or commutativity. A *law* is an equation between two terms built from variables and $\diamond$, understood as holding for all values of the variables. The commutativity law is an example of such a law, $x \diamond y = y \diamond x$. The *order* of a law is the number of $\diamond$ symbols on both sides together. ETP catalogues all 4,694 laws of order at most four, up to renaming variables and swapping the two sides, and represents syntactic tautologies by $x = x$.

Before comparing equations, we convert them into a standard form that preserves parentheses, the order of arguments, and which occurrences refer to the same variable. Renaming variables or swapping the two sides of an equality does not change this form. However, different equations in the ETP catalogue can still be logically equivalent.

A law S *implies* a law G, written $S \models G$, if every magma satisfying S also satisfies G. ETP has decided this for almost every ordered pair of catalogue laws, and we call an implication *certified* when ETP has a Lean proof that it holds or a verified counterexample magma showing that it does not. Conjectured results are not certified, and we do not use them. Given the original law S and the generated law G, the oracle looks up the two certified directions, $S \models G$ and $G \models S$, and assigns one of the four labels:

$$\begin{aligned} \text{Equivalent:} &\quad S \models G \text{ and } G \models S, \\ \text{Weaker:} &\quad S \models G \text{ and } G \not\models S, \\ \text{Stronger:} &\quad G \models S \text{ and } S \not\models G, \\ \text{Incomparable:} &\quad S \not\models G \text{ and } G \not\models S. \end{aligned}$$

The labels describe the reconstructed equation relative to the original. A weaker equation permits more magmas; a stronger equation permits fewer. If either implication direction lacks the evidence required to classify the pair, we report the relationship as unknown.

3.2 Coverage and recovery metrics

We measure how often a model reconstructs an equation with the same mathematical meaning as the original. We call this **certified recovery** and consider that a reconstruction is successful when the oracle establishes that the original and generated equations are equivalent.

For model m and representation r, we have the recovery rate as

$$R_{m,r} = \frac{\sum_{i \in A_{m,r}} \mathbf{1}[\mathrm{verdict}(S_i, G_{m,r,i}) = \mathsf{equivalent}]}{|A_{m,r}|}, \tag{1}$$

where S_i is the original equation, $G_{m,r,i}$ is the model's reconstruction, and $A_{m,r}$ is the set of responses received from model m for representation r , excluding content-filter and transport-error losses.

We report nine outcomes, grouped here into three types: four established semantic relationships (equivalent, weaker, stronger, and incomparable); three cases where the relationship cannot be established (unknown, off-catalogue, and unparseable or unmappable); and two response failures (content-filter and transport errors). Unknown, off-catalogue, and unparseable outputs do not count as successful recovery. A successful recovery comes only from certified equivalence. Outputs with unknown relationships, unsupported expressions, or parsing and mapping failures remain in the denominator, so a model cannot raise its score by producing answers the oracle cannot judge. As a consequence, $1 - R_{m,r}$ measures the fraction of responses without established equivalence and

not the fraction proven to have changed meaning. We report the distribution of all outcomes to differentiate between proven semantic differences and limitations of our evaluation procedure.

Finally, we use a fixed procedure to extract equations from model answers. Responses are stripped of formatting wrappers and we attempt to interpret the entire response. If the entire cleaned response is not accepted, we examine progressively smaller portions using a fixed sequence of extraction rules. The extractor never sees the reference equation. We also report a secondary *strict-extraction* score that accepts only the whole cleaned response in Appendix E.

3.3 Why merging variables can weaken a law

One way a language model can change a specification is by mistakenly treating two distinct variables as the same variable. This can make the resulting requirement easier to satisfy.

We observed such an error in our main experiment. When presented with the equation $\texttt{l} = \texttt{l} \diamond \texttt{I}$, whose two variables are distinct but look alike (l and I), Claude Opus 5 reconstructed it as $x = x \diamond x$. Using ordinary variable names, the original and reconstructed equations are:

$$S:\ x = x \diamond y, \qquad G:\ x = x \diamond x. \tag{2}$$

G is obtained by identifying y with x. The two-element magma in which $a \diamond b = b$ ("return the second argument") satisfies G but violates S whenever $x \neq y$. Hence this edit is strictly weaker, matching the stored certified verdict.

This example shows a general property of universally quantified equations. Let σ map variables to variables, possibly merging distinct ones. Since the original equation must hold for every assignment of its variables, it must also hold when some variables are assigned the same value. Therefore, $S \models \sigma(S)$, the original equation implies the equation obtained by merging variables. Such a substitution can weaken a requirement, although it does not necessarily make it strictly weaker. Strict weakening requires showing that the reverse implication does not hold.

4 Experimental design

We select 500 equations from ETP's catalogue of 4,694 laws. The equations differ in order, measured by the number of times the binary operation $\diamond$ appears. To include both simple and more complex equations, we use all 44 eligible laws of orders 1 and 2, randomly select 150 of the 364 order-3 laws, and 306 of the 4,284 order-4 laws. We exclude the two exceptional laws $x = x$ and $x = y$. Because this sample contains a larger proportion of simple laws than the full catalogue, we also report results adjusted for the catalogue's distribution of equation complexity.

For each selected equation, we generate 17 alternative representations and keep the original mathematical notation, which we call canonical infix, as the control, giving 18 representations in total. These include mathematical notations, programming languages, structured data, and English descriptions.

Each representation is produced by a deterministic converter, and each converter has an inverse reader. Reading back every generated representation of all 4,694 catalogue equations recovers the original equation structure in every case. For example, the equation $x = y \diamond (x \diamond y)$ becomes `= x <> y <> x y` in prefix notation, `x y x y <> <> =` in postfix notation, "`x equals y diamond open bracket x diamond y close bracket`" in bracket words, "`x equals the diamond of y and the diamond of x and y`" in nested English, and $\texttt{l} = \texttt{I} \diamond (\texttt{l} \diamond \texttt{I})$ with confusable names. Appendix A describes all eighteen.

Prompts and models. Each model receives one equation in one of the 18 representations, together with a *notation note*: a short explanation of how to read that representation, saying what the operation symbol is, what the variables are, and how grouping is written. We ask the model to write the equation in a common mathematical notation, using explicit parentheses and single lowercase letters for variables. Variable names may change, but the mathematical meaning should remain the same.

The same task instructions are used across representations, with a notation-specific explanation for each input. The original instructions include "Do not restate the input," which may conflict with the reconstruction task, particularly when the equation is already in the requested notation. We retain this protocol in the main experiment and separately test alternative instructions in Section 5.5. The

main recovery rates therefore characterize this task configuration, rather than intrinsic difficulty of the notations. Each model receives all 500 equations in all 18 representations, giving 9,000 calls per model. The prompts can be found in Appendix E and details on models in Appendix B.

Follow-up experiments. Four smaller experiments use a fixed subset of 150 of the 500 equations. Repeated sampling draws two further samples of every main call on those equations, giving three samples per item. The naming experiment changes only the variable names, under six schemes, for six models. The notation-note experiment changes only the note, under four conditions, for four models and four representations. The task-wording experiment changes only the instructions, under four variants, for seven models on canonical infix and LaTeX (8,400 calls). Each follow-up includes a fresh baseline on the same 150 equations and extends the main design.

Experimental runs and statistical comparisons. We first compare recovery rates across all representations and models. Because each model receives the same equations in different representations, we can also compare its performance on matched equations, rather than treating every response as an independent observation. Some calls return no usable response, so we use the 434 equations for which all model–representation combinations have responses when performing these matched comparisons. We test whether recovery differs across representations using Cochran's Q, and then compare each alternative representation with the original notation using McNemar tests. We adjust for multiple comparisons using Holm's method. We also check if the main patterns change when accounting for the uneven distribution of equation complexity in our sample. The reported confidence intervals summarize uncertainty for the evaluated conditions and do not establish that our findings generalize to other models or tasks.

Two additional draws on the 150-law subset give a median per-cell recovery range of 2.0 points (maximum 20.0), far smaller than the 65.2-point span; however, nine of seventeen adjacent pooled gaps fall below that median, so we interpret broad groups rather than fine ranks (Appendix D).

5 Results

5.1 Representation changes recovery

The form of an equation has a large effect on the recovery rate of reconstruction by language models. Figure 1 shows that certified recovery, averaged across the nine evaluated models, ranges from 73.7% for bracket-word descriptions to 8.5% for postfix notation, a difference of 65.2 percentage points. The canonical infix control, which already presents the equation in the requested output notation, achieves only 49.6% recovery. This result suggests that the task instructions themselves affect reconstruction and is investigated in Section 5.5. Appendix Table 4 contains further analysis. The common-subset span is similarly large. Post-stratification shifts individual estimates by at most approximately three points, preserving the broad pattern. For every model, Q rejects equality across notation conditions ($Q = 1,235.9$–$4,033.7$, $df = 17$, all $p < 10^{-12}$). Of 153 noncontrol model/representation comparisons, 123 differ significantly from control after within-model Holm adjustment.

Mechanical English is in the high-recovery group: bracket words score 73.7% and nested noun phrases 68.5%. The latter traverses terms in the same prefix order as symbolic Polish notation, which scores 16.9%. The 51.6-point contrast concerns the complete representation and its note, including English parsing scaffolding, rather than prefix order alone. Structural and formal-language representations show substantial model interactions (Figure 1). For example, Qwen recovers 98.0% from Lean but 44.4% from canonical infix; Mistral recovers no Lean laws across 500 calls. Opus 5 recovers approximately 53% on RPN (postfix) , while other models are below 10%. These interactions caution against choosing a format from pooled results alone.

5.2 The oracle distinguishes real semantic drift from coverage loss

A low recovery score does not necessarily mean that a model changed the equation's meaning. Some reconstructions are provably different, while others fall outside the comparison procedure's coverage. Of the 81,000 main responses, 42,151 are proven equivalent to the original. Another 16,368 are proven non-equivalent: 7,705 are strictly weaker, 1,087 strictly stronger, and 7,576 incomparable.

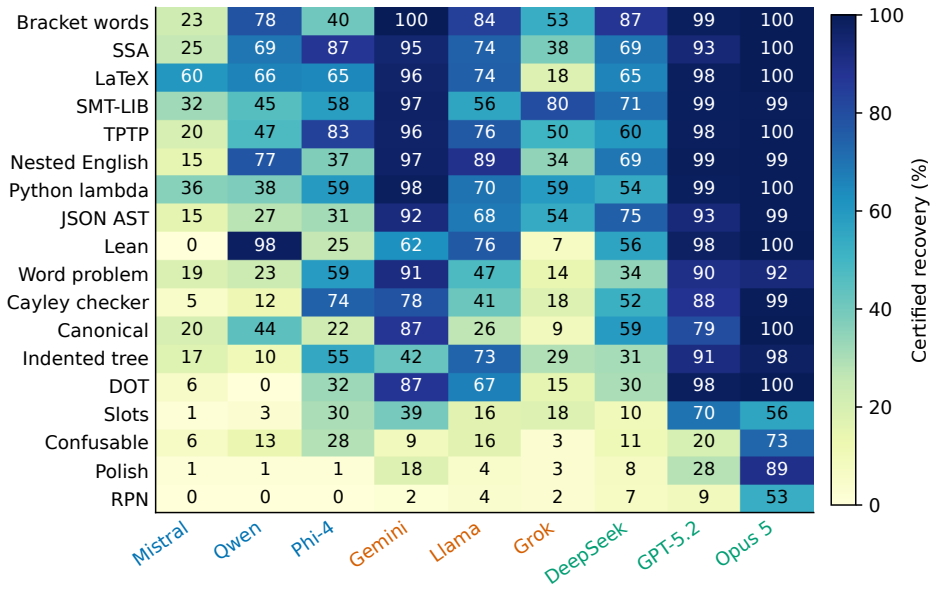


Figure 1: Main recovery by model and representation, with cells rounded to integers. Rows are ordered by pooled recovery; model labels are colored by descriptive tier (small: blue; medium: orange; frontier: green). The heatmap summarizes the fixed roster, not an inference about model scale. Missing responses are excluded within each cell.

The remaining outcomes include 19,472 outside-catalogue responses, 2,925 parsing or mapping failures, 15 unknown comparisons, and 69 content-filter losses. Figure **??** (Appendix C) shows this composition per representation.

Importantly, 2,281 equivalent reconstructions map to a different catalogue entry from the original. An evaluation based only on canonical-expression matching would incorrectly count these as failures. Visually confusable variable names are associated with a particularly consequential error: 1,365 of 4,500 reconstructions (30.3%) are proven strictly weaker than the original, compared with 370 of 4,500 (8.2%) under LaTeX notation.

Of these weaker reconstructions, 1,279 (93.7%) contain fewer distinct variables than the original, compared with 108 of 370 (29.2%) for LaTeX. This pattern is consistent with models merging variables that should remain distinct. However, the weakening classifications come from the certified implication results, independently of this variable-count measure. Equation 2 illustrates one instance where merging variables strictly weakens a law.

RPN (postfix) shows a different failure pattern. Its off-catalogue share is 2,434/4,500 (54.1%) responses. Inspection of recorded mapping reasons identifies 2,340/4,500 (52.0%) as valid magma identities above order four; 94 other outputs violate catalogue restrictions. Many well-formed responses therefore exceed the oracle's coverage. They cannot be certified by this oracle, but their larger syntax *does not prove inequivalence*. A translation evaluator needs both semantic outcomes and coverage statistics to avoid turning this measurement limitation into an unsupported error claim.

5.3 Visually similar variable names can cause models to lose variable distinctions

We next test if changing variable names alone affects reconstruction. We keep the equations, mathematical notation, model selection, and instructions fixed, changing only how variables are named. We test six naming schemes, including lowercase letters, indexed names, and visually similar characters from different alphabets (Figure 2). The renaming preserves which variables are distinct, so the underlying mathematical requirements remain unchanged.

Recovery falls from 64.4% with ordinary variable names to 3.1% with visually similar Latin and Cyrillic characters. Among incorrect reconstructions whose variables can be counted, the proportion

Table 1: Canonical-control framing probe (recovery %). The no-transform instruction adds an explicit prohibition on deriving consequences, generalizing, or transforming the identity. Bare removes the magma preamble. These are fresh 150-law baselines, not the main 500-law control estimates.

Model	Baseline	No-transform instruction	Bare	Both
Mistral Small	18.7	65.3	19.3	69.3
Qwen3 instruct	50.0	75.3	36.0	80.0
Phi-4	24.7	43.3	21.3	46.0
Llama 4 Maverick	25.3	68.7	16.7	77.3
Grok 4.20	9.3	15.3	28.7	82.0
Gemini 2.5 Flash	83.3	98.0	92.0	95.3
GPT-5.2	79.3	98.0	78.0	97.3

containing fewer distinct variables rises from 33.4% to 98.0%. This suggests that models frequently lose distinctions between variables, rather than simply failing to produce an answer. The homoglyph condition's read-back validation is discussed in Appendix H.

The aggregate trend is strong, but the proxy is not strictly monotone between confusable and repeat schemes, and schemes are not a calibrated numerical confusability scale. Opus goes from 100% on canonical letters to 6.0% on homoglyphs, despite retaining 74.1% on confusable ASCII and 88.0% on repeated-letter names. Indexed names improve Llama's recovery from 21.3% to 65.3%, and GPT's from 82.0% to 90.7%. Unusual or longer names are therefore not uniformly detrimental.

5.4 Explaining notation can improve reconstruction, but more text is not always better

To test if models struggle because they lack instructions for reading unfamiliar notation, we vary the explanation of each equation. We compare four conditions: no explanation, the original short explanation, a longer explanation with additional text, and a detailed explanation with reading rules and an example. Without any notation note, nested-English minus prefix recovery remains positive for all four models.

Reading content can nevertheless change the result dramatically. GPT's prefix recovery moves from 34.0% with the current note to 93.3% with a generous note; Opus's postfix moves from 49.3% to 94.0%. Both contrasts survive within-model Holm correction. Gemini's prefix recovery similarly rises from 19.3% to 68.7%; Appendix F gives all sixteen cells. Against length-matched padding, the mean gain across prefix/postfix cells is +27.8 points; across nested-English/LaTeX cells it is -0.2 points. The padding-minus-current mean is -2.2 points across all sixteen cells. These comparisons support a benefit from the *specific reading content*, not a general benefit from longer prompts.

More explanation can also hurt. Generous versus current LaTeX notes reduce Qwen by 15.3 points ($p_{\mathrm{Holm}} = 0.0055$) and Gemini by 14.7 ($p_{\mathrm{Holm}} = 0.00010$). The intervention bundles rules and examples; their separate effects are not identified, and padding is not guaranteed to be cognitively inert. These findings argue for notation-specific documentation rather than maximal explanation.

5.5 The canonical control reveals task-wording sensitivity

Canonical infix shows the identity in the requested output notation, yet pooled recovery is only 49.6%. This is a control for the *configured recovery task*, not an inherent model ceiling. The original output contract includes "Do not restate the input," which can conflict with recovering a supplied identity. A 2×2 framing probe adds an explicit no-transform instruction, removes the magma preamble, or applies both, on canonical and LaTeX inputs. Seven models and 150 laws produce 8,400 calls.

The sentence alone increases recovery by 46.7 points for Mistral and 43.3 for Llama (differences from unrounded counts). Grok's largest change, +72.7 points, requires *both* edits; its sentence-alone gain is only +6.0. Deleting the preamble alone is not significant for five of seven models. The no-transform instruction also improves LaTeX recovery: Mistral 60.0% to 78.0%, Qwen 78.0% to 96.7%, and Llama 74.7% to 94.7%, each adjusted $p < 0.001$. Improvement is therefore not specific to copying the canonical input. The probe supports instruction sensitivity and is consistent with over-transformation, but does not uniquely establish that psychological mechanism. It also prevents interpreting the main results as intrinsic notation difficulty independent of the recovery request.

6 Implications for verifiable coding and limitations

Specification preservation is a separate obligation. Within this domain, a certified weaker reconstruction admits magmas excluded by the original identity. A later proof of that reconstruction cannot restore the lost requirement. The right-projection magma $a \diamond b = b$ (return the second argument) in Equation 2 makes this gap concrete. More generally, equivalence of specifications and correctness of implementations against them are separate checks. We measure the former for small equations, not the latter for generated programs.

Keep verdict direction and coverage in the interface. A Boolean pass/fail conflates opposite inclusion errors with out-of-coverage answers. A preservation checker should expose intended-to-generated and generated-to-intended evidence, together with an explicit uncertified outcome. For supported pairs, weakening and strengthening have distinct downstream consequences. For unsupported pairs, a stronger backend or human assessment is needed before declaring equivalence or inequivalence. This is an implication of the measurement, not a deployed tool evaluation.

Identifiers and instructions deserve evaluation. The confusable study motivates preserving a bijection between source and target variable identities, checking co-reference structure, and considering existing Unicode identifier-security mechanisms [Unicode Consortium, 2025]. The prompt probes show that reading examples and explicit preservation instructions can help, but benefit is model/notation dependent. They were not tested jointly, and extra documentation consumes prompt tokens. Neither a discriminability linter nor an end-to-end repair pipeline is evaluated here.

Broader impact. Preservation checks could reduce false assurance when specifications are handled by assistants. Conversely, the identifier distortions studied here could be introduced deliberately to obscure variable distinctions. Treating an uncertified output as a proven error, or a passed small-equation check as a software-system guarantee, could also mislead users. We therefore report the oracle boundary explicitly and evaluate no deployed security exploit.

Scope and measurement threats. The dataset comprises single equations, with common algebraic notation possibly present in pretraining; contamination is not quantified. The strata overrepresent lower orders, and even reweighted estimates describe the catalogue, not software specifications in general. Models are a fixed historical API roster with unequal supported parameters and heterogeneous routing; tier comparisons are descriptive. Follow-ups reused the 150-law subset and dispatched tasks in condition blocks, leaving possible temporal/provider effects. Notation notes and narrative scaffolding differ across representations; the word problem repeats caveats already in the frame. The Unicode naming schemes vary script, visual similarity, length, and tokenization together.

The trusted measurement pipeline includes parsing, normalization, mapping, render/inverse checks, and correct consumption of certified matrix statuses. Some renderers/readers share helpers, so inverse agreement is not independent formal verification. The catalogue is incomplete for arbitrary output syntax, and strict extraction follows cleaning. Sixty-nine main filter losses are concentrated in particular representations; 499 main responses finish at the token limit, so truncation is not absent. These limitations narrow the interpretation of certified recovery, but do not negate the directly certified directional relations for the pairs that were matched

7 Conclusion

Our controlled experiments show that language models can turn different representations of the same mathematical equation into specifications with different meanings. By comparing the original and reconstructed equations using established implication results, we distinguish successful reconstructions from those that are provably weaker, stronger, or incomparable. We also identify cases where the available mathematical results cannot establish a relationship.

The measured recovery rates also depend on task instructions and explanations and not only on notation alone. Our experiments involve algebraic equations rather than realistic software specifications, and they demonstrate concrete failure modes without establishing how frequently they occur in practical coding workflows.

Proving that an implementation satisfies a generated specification does not establish that the original requirement was preserved. Checking for semantic equivalence, reporting the direction of any change, and explicitly reporting the limits of evaluation are necessary safeguards against false confidence. These findings reinforce the need to treat specification validation and implementation verification as separate because a correct proof cannot compensate for an incorrectly translated requirement.

Reproducibility and use of language models

Code for the oracle, converters, experiment runners, grading, and a preparation entry point with embedded prompt configuration, together with the saved experiment data underlying the reported tables, is archived at `https://doi.org/10.5281/zenodo.23392977`. Appendix I describes the release and gives setup instructions. Reported API costs total approximately $103.92 for the 153,000-call study; this is not a complete ledger of billed retries or pilot calls. The design record precedes the main experiment according to project documentation, but is not an immutable public preregistration. We disclose the model substitution, corrected denominator, and exploratory extensions in Appendix I.1.

LLMs are the evaluated systems. The earlier prose study additionally uses Qwen to generate descriptions (Appendix J); those descriptions are excluded from the main deterministic experiment. An LLM assistant was used to draft and edit this manuscript and to check selected numerical claims against stored artifacts. These steps did not generate new experimental results or machine-checked proofs.

Acknowledgments and Disclosure of Funding

We thank Apart Research and the Secure Program Synthesis Fellowship for compute funds and for providing the framework in which this work took place.

References

Anmol Agarwal, Natalie Neamtu, Pranjal Aggarwal, Seungone Kim, Jannis Limperg, Cedric Flamant, Kanna Shimizu, Bryan Parno, and Sean Welleck. Verus-SpecGym: An Agentic Environment for Evaluating Specification Autoformalization. *arXiv preprint arXiv:2605.26457*, 2026. `https://arxiv.org/abs/2605.26457`.

Matthew Bolan, Joachim Breitner, Jose Brox, Nicholas Carlini, Mario Carneiro, Floris van Doorn, et al. The Equational Theories Project: Advancing Collaborative Mathematical Research at Scale. *arXiv preprint arXiv:2512.07087*, 2025. `https://arxiv.org/abs/2512.07087`.

Nicholas Boucher and Ross Anderson. Trojan Source: Invisible Vulnerabilities. In *32nd USENIX Security Symposium*, pages 6507–6524, 2023. `https://www.usenix.org/conference/usenixsecurity23/presentation/boucher`.

Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. Bad Characters: Imperceptible NLP Attacks. In *IEEE Symposium on Security and Privacy*, 2022. `https://arxiv.org/abs/2106.09898`.

Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions? In *ACM International Conference on the Foundations of Software Engineering (FSE)*, 2024. `https://arxiv.org/abs/2310.01831`.

Minwoo Kang, Mingjie Liu, Ghaith Bany Hamad, Syed Suhaib, and Haoxing Ren. FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware. *arXiv preprint arXiv:2410.23299*, 2024. `https://arxiv.org/abs/2410.23299`.

Mikoláš Janota. Experimental Results for Vampire on the Equational Theories Project. *arXiv preprint arXiv:2508.15856*, 2025. `https://arxiv.org/abs/2508.15856`.

Logan Murphy, Kaiyu Yang, Jialiang Sun, Zhaoyu Li, Anima Anandkumar, and Xujie Si. Autoformalizing Euclidean Geometry. In *International Conference on Machine Learning*, 2024. https://arxiv.org/abs/2405.17216.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying about Prompt Formatting. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2310.11324.

Unicode Consortium. Unicode Security Mechanisms. Unicode Technical Standard #39, revision 32, Unicode 17.0.0, 2025. https://www.unicode.org/reports/tr39/tr39-32.html.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with Large Language Models. *arXiv preprint arXiv:2205.12615*, 2022. https://arxiv.org/abs/2205.12615.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A Cross-System Benchmark for Formal Olympiad-Level Mathematics. In *International Conference on Learning Representations*, 2022. https://arxiv.org/abs/2109.00110.

A Representation definitions and validation

We use the `2024-11-10` ETP outcomes snapshot. Its $4,694^2$ directed entries comprise 8,178,279 certified true and 13,855,167 certified false entries, plus 186 conjectured false and four unknown entries. Thus the local snapshot has 190 uncertified directions, although the later ETP project reports completing the graph [Bolan et al., 2025]. Same-law outputs are immediately equivalent. The experiment consumes certified outcome labels; it does not replay proof certificates on each query.

Each catalogue stores `node`, `formal`, operation order, distinct-variable count, and representation-specific text, with generator/style and validation metadata. Canonical infix is taken from the formal field of an alternate catalogue. The alternate notations are:

1. **LaTeX**: infix `\diamond` and explicit parentheses.
2. **Lean**: universally quantified proposition over a type with a `Magma` operation; the recovery body omits numbered law identifiers.
3. **Python lambda**: `lambda op, x, ...: lhs == rhs`, with nested `op(a, b)` calls and explicit variable-parameter order.
4. **Python Cayley**: `def law(table)`, local `n = len(table)`, and `all(... for x in range(n) ...)` with nested `table[a][b]` lookups.
5. **Bracket-word text**: infix words `diamond`, `equals`, `open bracket`, and `close bracket`, with variable letters retained.
6. **Nested-English text**: recursive `the diamond of A and B` phrases, with `equals` between sides.
7. **SSA**: post-order intermediate definitions, followed by equality of side references; read-back expands definitions and checks their order.
8. **Slots**: a pair of nested binary lists; integer leaves are first-appearance variable indices, preserving repeated-variable co-reference.
9. **RPN**: variable/postfix-operation tokens for both terms and a final equality token; inverse reading simulates ordered stack consumption.
10. **prefix**: equality followed by two prefix terms; every binary operation consumes two recursive terms.
11. **TPTP**: universally quantified first-order equality with uppercase variable names and nested `op(A, B)` applications; formula name is generic.
12. **SMT-LIB**: uninterpreted sort/operation declarations plus a universally quantified equality assertion using nested `(op a b)` terms.
13. **Indented tree**: one equality-rooted tree with two ordered children and four-column indentation; branch markers carry traversal structure.
14. **DOT**: labeled equality/operation/variable nodes and ordered edges in textual Graphviz source, using `ordering=out`.
15. **JSON AST**: one-key equality object with a two-term list; variables are strings and operations are one-key diamond objects with ordered arguments.
16. **Confusable names**: look-alike names through canonical infix with a bijective renaming to `l I ll lI Il II`.
17. **Word problem**: a fixed welding scenario and recursively nested "part made by welding A to B" descriptions, universally quantified over parts.

Readers recover AST structure through format-specific parsing. Newer text and structural readers recover canonical infix and invoke the oracle parser; Python readers use Python's AST module. Confusable read-back is compared after bijective normalization, other catalogue checks use exact structure. Builders also check canonical source forms and, where implemented, distinct rendered strings across nodes. Converter self-tests include malformed/corrupted renderings. These mechanisms establish deterministic fidelity on the recorded catalogue under the tested decoder, not independent correctness of arbitrary external parsers or proof-assistant elaboration.

B Full endpoint configuration and model summaries

Table 2: Historical OpenRouter endpoint IDs and parameters. All calls cap output at 900 tokens. "No field" means the runner does not set a reasoning argument. The tier is a project-defined category.

Tier	Endpoint	Reasoning	Temperature / seed
Small	`mistralai/mistral-small-3.2-24b-instruct`	No field	0 / yes
Small	`qwen/qwen3-30b-a3b-instruct-2507`	No field	0 / yes
Small	`microsoft/phi-4`	No field	0 / yes
Medium	`google/gemini-2.5-flash`	Disabled	0 / yes
Medium	`meta-llama/llama-4-maverick`	No field	0 / yes
Medium	`x-ai/grok-4.20`	Disabled	0 / yes
Frontier	`deepseek/deepseek-v3.2`	Disabled	0 / yes
Frontier	`openai/gpt-5.2`	Effort none	Omitted / yes
Frontier	`anthropic/claude-opus-5`	Disabled	0 / no

Supported seeds use `20260830 + k`; $k = 0$ for the main and follow-up calls, $k = 1, 2$ for extra repeats. The main task shuffle seed is 771 plus model roster index. Sample seed is 20260830, repeat-subset selection seed 20260831. Project documentation reports an 80-call reasoning preflight for the original roster; DeepSeek's explicit disable setting was added after Kimi substitution. Stored graded main/repeat rows record zero reasoning tokens. Explanatory ordinary completion text is still possible.

Table 3: Recovery and strict-extraction scores (%), with all 9,000 expected main calls per model in the denominator to make loss visible. These differ from Equation 1 for a model with excluded missing responses. The canonical column uses all-answered cell accuracy.

Model	Recovery / expected calls	Strict / expected calls	Canonical
Phi-4	43.7	7.3	22.2
Mistral Small	16.7	16.6	19.8
Qwen3 instruct	36.2	36.2	44.4
Gemini 2.5 Flash	71.5	71.5	87.0
Llama 4 Maverick	53.5	48.7	26.2
Grok 4.20	27.9	27.9	9.0
Opus 5	91.3	82.5	100.0
DeepSeek V3.2	47.0	46.9	58.8
GPT-5.2	80.5	80.5	79.4

The strict gap is 36.4 points for Phi and 8.8 for Opus, rather than a single outlier. Model tiers should not be interpreted as controlled scale effects. Open-weight versus closed-weight recovery is likewise confounded with the selected roster and tier distribution.

C Exact main counts and supplementary analyses

Main filter losses by representation are JSON AST 44, confusable names 20, SSA two, nested-English two, and Cayley one. There are 499 length finishes, approximately 0.62% of main calls; a length finish may still yield an extracted answer. The main/repeat total includes 99 filtered rows. Response loss is distinct from the much larger coverage/parse-failure rate.

Difficulty covariates. An exploratory logistic model of correctness uses representation, order, and tier, with standard errors clustered by source law: 80,931 responses, 500 clusters. The order coefficient is -0.0169 ($p = 0.566$). Representative coefficients relative to canonical are bracket words $+1.2263$, SSA $+1.1414$, LaTeX $+1.0940$, slots -1.1779, confusable -1.6308, prefix -1.8445, and postfix -2.6883. Frontier versus small tier is $+2.1907$; this is a log-odds association, not directly comparable with percentage-point intervention gains.

Depth 1/2/3/4 descriptive recovery is 55.4/58.8/51.0/45.4%; recovery for 1–5 distinct variables is 55.6/52.8/51.4/52.4/52.4%. Depth is not included in that fitted regression, so the depth association is

Table 4: Main certified recovery (%), pooled over the fixed nine-model roster. All-answered estimates exclude only missing-response outcomes. Common uses 434 globally complete laws; post-stratified reweights eligible catalogue orders. Fine rank differences should not be read as established pairwise superiority. Exact counts and nominal intervals are in Appendix C.

Representation	All answered	Common	Post-stratified
Bracket words	73.7	73.6	73.3
SSA	72.2	72.4	71.7
LaTeX	71.4	71.6	73.0
SMT-LIB	70.7	70.8	69.7
TPTP	70.1	70.0	68.8
Nested English	68.5	69.1	67.0
Python lambda	67.9	67.8	67.2
JSON AST	61.2	62.6	59.9
Lean	58.1	58.5	61.1
Word problem	52.2	53.3	52.2
Cayley checker	51.9	52.5	51.2
Canonical infix	49.6	49.5	50.6
Indented tree	49.5	50.4	50.6
DOT graph	48.4	48.0	46.7
Integer slots	26.8	27.5	28.1
Confusable names	19.7	20.3	19.5
Polish (prefix)	16.9	17.5	16.7
RPN (postfix)	8.5	8.3	7.9

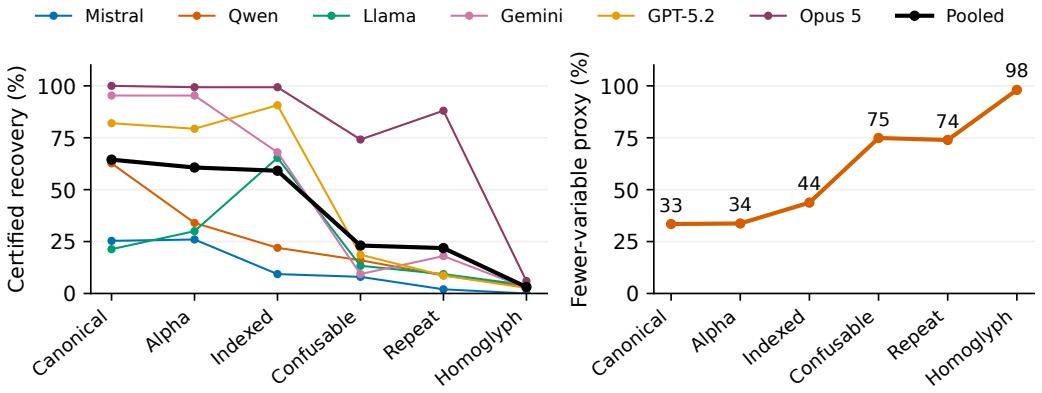


Figure 2: Identifier study over six models and 150 laws. Left: recovery per model and pooled over models, excluding 11 filtered responses. Right: fewer distinct variables among incorrect outputs whose variables can be counted; this is a proxy, not a proof of a pure variable-merge edit. Schemes are categorical and not a calibrated numerical confusability scale. More details are provided in Appendix H.

unadjusted. Equivalence-class size buckets unique/2–9/10+ have 54.0/54.2/51.0% recovery. These are exploratory descriptive comparisons, not evidence that any covariate has no causal effect.

D Repeated sampling

On the 150 repeated laws, all three samples have the same reporting outcome in 18,704/24,300 triples (77.0%). Agreement refers to *outcome labels*, including coverage failures. Across 162 model/representation cells, the range of recovery over three draws has median 2.0 points, mean 2.9, 90th percentile 6.6, and maximum 20.0. The broad 65.2-point main span is much larger than typical observed resampling variation. However, nine of seventeen adjacent pooled gaps are smaller than the median cell range. We emphasize broad groups rather than a best-format claim.

Table 5: Pooled main certified-equivalent numerators, nonmissing-response denominators, and nominal Wilson 95% intervals. Intervals are descriptive binomial summaries for the fixed roster and do not incorporate all dependence between repeated laws or uncertainty over model selection.

Representation	Equivalent	Nonmissing	95% interval
Bracket words	3317	4500	72.4–75.0
SSA	3249	4498	70.9–73.5
LaTeX	3213	4500	70.1–72.7
SMT-LIB	3182	4500	69.4–72.0
TPTP	3155	4500	68.8–71.4
Nested English	3081	4498	67.1–69.8
Python lambda	3057	4500	66.6–69.3
JSON AST	2729	4456	59.8–62.7
Lean	2613	4500	56.6–59.5
Word problem	2349	4500	50.7–53.7
Cayley checker	2334	4499	50.4–53.3
Canonical infix	2234	4500	48.2–51.1
Indented tree	2229	4500	48.1–51.0
DOT graph	2176	4500	46.9–49.8
Integer slots	1206	4500	25.5–28.1
Confusable names	882	4480	18.5–20.9
Polish (prefix)	761	4500	15.8–18.0
RPN (postfix)	384	4500	7.8–9.4

Table 6: Main outcome counts over all 4,500 expected responses per representation. E/W/S/I are equivalent/weaker/stronger/incomparable; U is unknown; O is off catalogue; P is unparseable/mapping failure; F is content filter. No main output has the transport-error outcome.

Representation	E	W	S	I	U	O	P	F
Bracket words	3317	239	27	220	2	622	73	0
SSA	3249	345	51	333	0	425	95	2
LaTeX	3213	370	31	289	0	569	28	0
SMT-LIB	3182	267	124	318	2	412	195	0
TPTP	3155	335	58	354	0	485	113	0
Nested English	3081	270	34	299	1	700	113	2
Python lambda	3057	326	82	363	0	525	147	0
JSON AST	2729	271	33	348	0	911	164	44
Lean	2613	385	39	297	0	1045	121	0
Word problem	2349	331	46	478	3	1225	68	0
Cayley checker	2334	603	76	591	3	819	73	1
Canonical infix	2234	666	55	550	1	954	40	0
Indented tree	2229	350	38	393	0	1327	163	0
DOT graph	2176	385	76	501	0	1323	39	0
Integer slots	1206	482	92	530	0	1930	260	0
Confusable names	882	1365	53	537	2	1527	114	20
Polish (prefix)	761	376	85	612	1	2239	426	0
RPN (postfix)	384	339	87	563	0	2434	693	0

The range is a descriptive statistic from three draws on a smaller subset, not a confidence interval or a universal two-point resolution threshold. Likewise, overlap of nominal marginal intervals does not establish pairwise equivalence. Confusable naming, slots, and postfix have the lowest outcome agreement (64.6%, 67.3%, 68.6%); SSA and bracket words have the highest (85.7%, 83.6%). Missing seed support and provider routing remain sources of uncontrolled variation.

E Extraction and exact baseline frame

Cleaning removes paired thinking/reasoning tags, selected stray tags, typesetting spacing, math delimiters, and selected LaTeX text wrappers. The extraction acceptance rule in the implemented grader is mapper status different from `parse-failure`. This is broader than requiring successful identity parsing: an outside-fragment or internal-error candidate can terminate the ladder before a

later valid candidate. Off-catalogue includes constants, unsupported relation symbols, or additional operations as well as valid above-order-four identities. The reported results retain this original implementation; no repaired-extractor experiment is presented.

The strict-extraction score requires `exact` after cleaning and oracle normalization. It can therefore accept alternate operator notation or cleaned presentation that does not literally follow the raw output contract. The visible-CoT field is an extraction/presentation heuristic; it does not measure hidden reasoning expenditure, and we do not use it to infer an internal reasoning mechanism.

The following reproduces the baseline frame, with mathematical glyphs rendered as TeX and placeholders shown explicitly. For canonical input, the note is: "The operation is written $\diamond$ between its two arguments. Variables are single lowercase letters. Grouping is shown by parentheses around each nested operation." For confusable naming: "The operation is written $\diamond$ between its two arguments. Variables are the identifiers; different identifiers are different variables. Grouping is shown by parentheses."

> A single identity that holds in a magma is written below. Recover the identity itself.
>
> The identity involves one binary operation and holds for all values of its variables. The operation is on an arbitrary set: it is not assumed to be associative or commutative, and there is no identity element.
>
> *[Notation note]*
>
> Requirements:
>
> - Output only the identity, on one line, in the form: left side = right side.
> - Write the operation as $\diamond$ and make every grouping explicit with parentheses: write $x \diamond (y \diamond z)$ or $(x \diamond y) \diamond z$, never $x \diamond y \diamond z$.
> - Use single lowercase letters for variables; you may choose any letters.
> - Do not restate the input, do not name the identity, do not explain, and do not add any other text.
>
> *[Rendered law]*

The anti-transformation probe inserts: "Write exactly the identity shown below. Do not derive a consequence of it, do not generalise it, and do not transform it in any way." The bare-frame manipulation removes the magma preamble paragraph above. Both retains the inserted sentence and removes that paragraph.

F Complete notation-note ablation

Table 7: All sixteen model/representation cells across four note conditions (recovery %). Denominator is 150 except Opus generous nested-English 139, Opus generous prefix 149, and Opus padded prefix 149, because filtered responses are excluded. These exclusions matter for interpreting apparent ceiling scores.

Model	Representation	None	Current	Padded	Generous
Qwen3 instruct	Nested English	64.0	79.3	60.7	84.0
Qwen3 instruct	Polish (prefix)	0.0	0.7	0.7	15.3
Qwen3 instruct	RPN (postfix)	0.7	0.0	0.7	3.3
Qwen3 instruct	LaTeX	64.0	72.7	69.3	57.3
Gemini 2.5 Flash	Nested English	94.7	96.0	98.0	98.0
Gemini 2.5 Flash	Polish (prefix)	4.7	19.3	28.7	68.7
Gemini 2.5 Flash	RPN (postfix)	6.0	2.0	4.0	20.0
Gemini 2.5 Flash	LaTeX	90.7	96.0	96.0	81.3
GPT-5.2	Nested English	93.3	98.7	98.7	98.7
GPT-5.2	Polish (prefix)	10.7	34.0	30.0	93.3
GPT-5.2	RPN (postfix)	8.0	4.7	8.7	34.7
GPT-5.2	LaTeX	86.0	96.7	95.3	92.7
Opus 5	Nested English	98.0	98.0	99.3	100.0
Opus 5	Polish (prefix)	54.7	90.7	67.8	82.6
Opus 5	RPN (postfix)	36.0	49.3	48.7	94.0
Opus 5	LaTeX	100.0	100.0	96.0	100.0

Per-representation note lengths for current/padded/generous are nested-English 29/78/76, prefix 35/106/101, postfix 35/84/87, and LaTeX 27/76/76 tokens under `cl100k_base`. Padded notes append complete on-topic sentences without additional parsing instructions or imperatives. Generous notes specify argument order and recursion/stack behavior, plus an example. For example, prefix explains that leading equality consumes two terms and each operation consumes two recursive terms; postfix explains pushes, two-value pops with left/right order, and final equality. The ablation cannot separate reading instructions from the included example.

The reported note tests compare none/padded/generous with current, paired by law and corrected within model across representation/condition contrasts. The descriptive generous-minus-padded mean is not itself a new family of Holm-corrected significance tests. Thirteen responses are filtered and 207 finish at the output cap. Notes and raw responses are retained in the existing artifacts.

G Exact notation notes and intervention text

The following are the recorded main notes. Mathematical glyphs are typeset rather than copied as Unicode characters; words and punctuation are unchanged. All notes are constant across laws. Their `cl100k_base` lengths appear in parentheses. The shared frame is in Appendix E.

`ascii_tree` (37 tokens) Each ◇ is one operation. Variables are the lowercase letters at the leaves. Grouping is shown by indentation: a node's two children are its arguments, the left one first.

`confusable_vars` (29 tokens) The operation is written ◇ between its two arguments. Variables are the identifiers; different identifiers are different variables. Grouping is shown by parentheses.

`etp_canonical` (28 tokens) The operation is written ◇ between its two arguments. Variables are single lowercase letters. Grouping is shown by parentheses around each nested operation.

`graphviz` (35 tokens) This is Graphviz DOT. Each ◇ node is an operation. Variables are letters in node labels. A node's two out-edges are its arguments, left first.

`json_ast` (31 tokens) This is JSON. The operation is an object keyed ◇ holding two arguments. Variables are the strings. Grouping is shown by how the objects nest.

`latex` (27 tokens) This is LaTeX. The operation is written \diamond between its two arguments. Variables are single letters. Grouping is shown by parentheses.

`lean` (31 tokens) This is Lean. The operation is written ◇ between its two arguments. Variables are the ones bound by the ∀. Grouping is shown by parentheses.

`polish` (35 tokens) The operation is ◇, written before its two arguments. Variables are single lowercase letters. Grouping is shown by position: each ◇ takes the two terms after it.

`py_cayley_table` (35 tokens) This is Python. The operation is the lookup table[a][b]. Variables are the names quantified in the comprehension. Grouping is shown by how the lookups nest.

`py_lambda` (33 tokens) This is Python. The operation is the call op(a, b). Variables are the lambda's parameters after op. Grouping is shown by how the calls nest.

`rpn` (35 tokens) The operation is ◇, written after its two arguments. Variables are single lowercase letters. Grouping is shown by position: each ◇ takes the two values before it.

`slots` (39 tokens) The operation is a two-element list. Variables are numbers, and equal numbers are the same variable. Grouping is shown by how the lists nest; the outermost list holds the two sides.

`smtlib` (36 tokens) This is SMT-LIB. The operation is the application (op a b). Variables are the names bound by the forall. Grouping is shown by how the applications nest.

`ssa` (36 tokens) The operation is written ◇ between its two arguments. Variables are single lowercase letters; names starting with t are intermediate results. Grouping is shown by which results each line uses.

`text` (30 tokens) The operation is the word diamond between its two arguments. Variables are single lowercase letters. Grouping is shown by the words open bracket and close bracket.

`text2` (29 tokens) The operation is the phrase 'the diamond of A and B'. Variables are single lowercase letters. Grouping is shown by how those phrases nest.

`tptp` (38 tokens) This is TPTP. The operation is the term op(A, B). Variables are the upper-case names bound by the quantifier. Grouping is shown by how the terms nest.

`word_problem` (32 tokens) The operation is welding one part to another, in that order. Variables are the lowercase letters naming parts. Grouping is shown by which part each welding uses.

Generous notes. These replace, rather than append to, the corresponding current note.

`text2` Each operation is the phrase 'the diamond of A and B', where A is the left argument and B the right. Variables are single lowercase letters. The phrases nest: in 'the diamond of x and the diamond of y and z', the left argument is x and the right argument is the whole inner phrase. Resolve the outermost phrase first, then each argument in turn.

`polish` This is Polish, or prefix, notation. The leading '=' takes the two terms that follow it as the left and right sides. Each '$\diamond$' takes the two terms that follow it as its left and right arguments. Variables are single lowercase letters. For example, '= $\diamond$ x y z' means (x $\diamond$ y) = z, because the '=' takes '$\diamond$ x y' as its first term and 'z' as its second. Every symbol consumes exactly two following terms.

`rpn` This is reverse Polish, or postfix, notation. Read left to right keeping a stack: a variable is pushed; a '$\diamond$' pops the two most recent values and pushes their combination, the earlier-pushed value being the left argument; the final '=' pops two values to form the two sides. Variables are single lowercase letters. For example, 'x y $\diamond$ z =' means (x $\diamond$ y) = z.

`latex` This is LaTeX. The operation is written \diamond between its two arguments, so 'x \diamond y' has x on the left and y on the right. Variables are single letters. Parentheses show grouping: 'x \diamond (y \diamond z)' has x on the left and the whole parenthesised expression on the right. Resolve the outermost operation first.

Padded notes. The recorded padded notes retain the current note and append the following sentences in order, stopping when the target generous-note token count is reached or when the next complete sentence would increase the absolute token count difference. No sentence is truncated. Thus the selected suffix, not only the token budget, is reproducible.

1. The operation combines exactly two arguments, and its result is again a value of the same kind.
2. The same operation is used at every point in the expression.
3. The identity is stated once, and it holds for every assignment of values to its variables.
4. Nothing in the expression below is abbreviated or elided.
5. The expression uses no convention beyond the one described above.
6. This is one identity, not a system of several.
7. The operation has no special cases, and the same reading applies throughout.

The none condition removes the note; repeated blank lines left by that removal are collapsed. The naming study uses the confusable note above for every scheme, including canonical. The framing probe's inserted sentence and deleted paragraph are specified in Appendix E.

H Identifier details and denominators

Homoglyph names are the six distinct strings with code points `U+0061`, `U+0430`, `U+0065`, `U+0435`, `U+006F`, `U+043E`, respectively: Latin small a, Cyrillic small a, Latin small e, Cyrillic small ie, Latin small o, Cyrillic small o. Code points are given instead of relying on this paper's font to make differences visible. The renderer substitutes names on AST variable leaves; distinct names remain distinct in the input construction.

The homoglyph input is an injective AST-variable substitution, but cannot be read back through the ASCII-oriented output parser. Requested outputs use single lowercase letters; unsupported echoed glyphs can fail parsing. We report this asymmetry rather than claiming the Unicode condition has the same end-to-end validation as the seventeen alternate catalogues.

Pooled recovery numerator/denominator counts by scheme are canonical 580/900, alpha 546/900, indexed 532/900, confusable 207/897, repeat 195/892, and homoglyph 28/900. Proxy denominators in Figure 2 require an incorrect extracted identity whose variables can be counted; unparseable and filtered outputs do not enter them. Thus "98%" is not a claim about all wrong calls. The same confusable notation note is used at every naming level, including canonical; its baseline differs from the main canonical condition's note.

Table 8: Identifier study: six models, 150 laws, 5,400 calls. Accuracy excludes 11 filtered responses; the fewer-variable proxy excludes wrong outputs whose variables cannot be counted. Its counts are shown explicitly. Homoglyph names pair Latin and Cyrillic letters; exact code points are in Appendix H.

Scheme	Identifiers	Recovery	Fewer vars.	Unparsed
Canonical	`x y z w u v`	64.4	33.4 (106/317)	0.8
Alpha	`a b c d e f`	60.7	33.7 (119/353)	0.6
Indexed	`v1 v2 ... v6`	59.1	43.8 (161/368)	0.1
Confusable	`l I ll lI Il II`	23.1	74.9 (510/681)	0.9
Repeat	`l ll ... llllll`	21.9	73.9 (496/671)	3.0
Homoglyph	Latin/Cyrillic a, e, o pairs	3.1	98.0 (853/870)	2.0

I Artifacts, execution flow, and deviations

Code and data release. The release (`https://doi.org/10.5281/zenodo.23392977`) contains Python and shell source for oracle parsing, mapping, matrix construction and comparison; deterministic converters and catalogue builders; sampling, prompt rendering, API collection and grading; follow-up runners; and earlier prose/implication experiment utilities. It also bundles the saved experiment data underlying the reported tables. It contains no repository history, local environments, or credentials.

The root `prepare_experiment.py` embeds the original frame and all eighteen notation notes. Its module documentation lists dependencies and execution commands. After installing those dependencies and supplying the upstream ETP `equations.txt` and `2024-11-10-outcomes.json.zip` through `ETP_EQUATIONS`/`ETP_OUTCOMES` (or `ETP_ROOT`), run:

```
python3 prepare_experiment.py
cd semantic-diff-test
python3 run_experiment.py --smoke 2
```

Preparation regenerates catalogue renderings, the local matrix, the sample, and prompt files without calling model APIs. Collection requires `OPENROUTER_API_KEY` and incurs API charges. Earlier experimental utilities need their separately generated datasets and upstream inputs.

Full research record. The following inventory describes the full project, including data and analysis artifacts omitted from the release:

- `oracle/`: normalizer, mapper, matrix builder and comparison API/CLI; local matrix and metadata for the historical outcomes snapshot.
- `translate/<representation>/`: deterministic converter, inverse reader, catalogue builder and generated 4,694-laws catalogue.
- `semantic-diff-test/sample.json`, `frame.txt`, `prompts.json`, and per-representation notes/templates/examples.
- Per-representation `raw-<model>.jsonl`: model/representation/node/draw keys, content, provider, finish, usage, retry count and timing.
- `results/graded.json`: extracted identity, semantic outcome, generated-node evidence where mapped, source-law covariates and run metadata; `results/summary.json` and `tables.md`: derived analyses.

- `ablation/`, `control_probe/`, `dose_confusable/`: follow-up raw rows and summaries; `figures/`: generated raster plots.

Core parsing uses Python's standard library. API access uses the OpenAI SDK against OpenRouter; analysis uses NumPy, SciPy, Matplotlib and `tiktoken`, with pandas/statsmodels for regression. The mapper requires ETP `equations.txt`, selected by `ETP_EQUATIONS` or `ETP_ROOT`; matrix construction additionally requires the snapshot outcomes archive.

Once those inputs and alternate catalogues are available, the full research workflow is as follows; `analyze.py` belongs to the original project and is not included in the code release:

```
cd semantic-diff-test
python3 build_sample.py
python3 render_prompts.py
python3 run_experiment.py --phase main
python3 run_experiment.py --phase repeat
python3 grade.py
python3 analyze.py
python3 ablate_notes.py --run --report
python3 probe_control.py --run --report
python3 dose_confusable.py --run --report
```

Commands launching experiments require an API key and incur spend. Regrading and analysis can instead use stored responses. The numerical checker targets selected fields in the Markdown analysis, not every sentence of this paper. No byte-identical rerun guarantee is implied for dynamic API providers.

Main checkpoints flush each completed row and check headers pinning model, temperature, output cap, reasoning, sample seed/count, and shared-frame hash. They do not pin all individual note/rendering hashes or the entire node list; follow-up checkpoints lack those identity headers. Error rows can be replaced by recovered calls, but deduplication otherwise keeps the first accepted keyed row rather than a general last-row rule. Artifact preservation is therefore important; changing prompts and resuming in place is not fully protected.

Final-row costs are main/repeat $79.6470, four-condition note ablation $18.7839, framing $1.3787, naming $4.1103, approximately $103.92 total. This accounting excludes the abandoned model, earlier experiments, and any billed discarded retry responses not preserved in returned row costs.

I.1 Changes to the recorded design

The recorded main design initially used `moonshotai/kimi-k2` for an open-weight frontier slot. Provider rate limiting prompted replacement with `deepseek/deepseek-v3.2`. The 533 partial Kimi rows are retained separately and excluded from all study tables. Its recorded final-response costs total approximately $0.1373.

The original denominator excluded unparseable and off-catalogue answers. We instead count them as unsuccessful certification, excluding only missing response outcomes; the original globally-gradeable rule yields zero common laws, whereas the implemented nonmissing rule yields 434. The original cap of 400 tokens was increased to 900; concurrency also changed from early estimates. Paired tests use the common subset, while pooled headline estimates use all nonmissing responses. The note ablation, later padded-note addition, framing probe, identifier study, and additional covariate analyses are follow-ups to the main observations. The design record is not claimed to be an immutable external preregistration.

J Earlier exploratory studies

These studies motivated the controlled recovery design but are not part of its 153,000-call ledger and do not provide independent replication of its effects.

Generated-English round trip. Qwen `qwen/qwen3-30b-a3b` generated descriptions for the 4,694 laws, with 4,612 usable descriptions, 42 empty and 40 failed results, at recorded cost $2.7783.

The same endpoint reconstructed from descriptions alone; of 4,694 output laws, 3,770 had usable answers. Semantic labels were 3,129 equivalent, 206 weaker, 48 stronger, 214 incomparable; 173 were unresolved and 924 lacked usable reconstructions. Equivalent share was 87.0% of resolved, 83.0% of answered, and 66.7% of all catalogue laws. Recorded reverse cost was $5.7867. This joint forward/reverse measurement confounds prose loss with reconstruction and has a substantial empty-answer tail. Its model/settings differ from the later instruct-2507 condition, and reasoning headroom differs from the controlled recovery protocol.

Implication judgment. A balanced sample of 250 certified true/false ordered pairs, seed 20260827, was presented in ETP, LaTeX, Lean, generated English, Python lambda and Cayley form to Mistral Small, Gemini 2.5 Flash and Opus 4.5. ATP time from a 2025-08-11 Vampire run provided difficulty signals, contextualized by Janota [2025]; truth labels used the historical oracle matrix. The dataset used 298 distinct equations, excluded nodes 1/2, required usable representations, and capped node reuse at four. Models answered TRUE/FALSE with brief justification and confidence.

There were 4,500 judgments, no unrecovered transport failures, and 4,426 scored answers; 74 lacked parsed verdicts, attributed to truncation and excluded from accuracy. Overall accuracies were 55.9/75.8/82.4% for the three models, recorded cost $23.17. On the global 197-pair common subset, all six notations agreed in 43.1/40.6/79.7% of laws. Model-specific omnibus p-values were 0.000389/0.0210/0.0446; only three of 45 pairwise contrasts survived within-model Holm adjustment. There was no same-prompt repeat arm, so agreement does not isolate notation effects from resampling.

Generated English pooled accuracy was 68.0%, versus Cayley 74.6%. Faithful-description subsets improved Gemini/Opus by about ten points, indicating a prose-fidelity confound. These English results concern model-written descriptions, unlike the deterministic English notation in the main study. Signal-detection analysis found substantial answer-bias shifts: Mistral favored FALSE and Opus favored TRUE. This stage addresses reasoning/criterion effects over an implication question, not recovery of a single law.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Sections 1–5 restrict claims to the configured recovery task and distinguish certified semantic disagreement from oracle coverage loss.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 and Appendices E and I.1 discuss scope, prompt ambiguity, parser limits, API variability, and exploratory follow-ups.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Section 3 states universal quantification and proves the variable-identification implication using induced assignments, with an explicit counterexample establishing strictness for Equation 2.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 3 and 4, and Appendices A, B, E, and G, specify the oracle snapshot, sampling, encodings, parameters, extraction, frame, and exact notes. Hosted-model variation prevents a byte-identical new API run; the artifacts described in Appendix I support verification from saved responses.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in

some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Code and the saved experiment data are archived at `https://doi.org/10.5281/zenodo.23392977`, with setup and execution instructions in Appendix I. Hosted-model variation prevents byte-identical reruns of API collection.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 and Appendices B and G specify inference parameters, sampling, prompts, and interventions; no model training is performed.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Sections 4 and 5 report paired tests, Holm corrections, and repeated-sampling variability. Appendix C explains nominal Wilson intervals and dependence limitations; exploratory naming and padding comparisons are not claimed as independently confirmed causal effects.

   Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: Appendix I reports API call counts, costs, providers, and output caps, but provider hardware, energy, and discarded-retry compute are unavailable, and a complete local-resource inventory is not documented.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The experiments use synthetic mathematical identities, without human participants or personal data. Prior work is credited and coverage limits, protocol changes, and remaining artifact-access and licensing-documentation gaps are disclosed.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 6 discusses potential reductions in false assurance, deliberate identifier distortion, and misleading extrapolation beyond the oracle domain.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases no trained model, sensitive scraped dataset, or demonstrated deployed exploit; evaluated inputs are small mathematical identities.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: ETP and prior work are credited, and snapshot/model versions are identified, but the research records do not establish a complete license and API-terms inventory for every asset. No completed audit is claimed.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The archived release (`https://doi.org/10.5281/zenodo.23392977`) bundles code, embedded prompt configuration, a documented preparation entry point, and the saved experiment data (Appendix I). It is released under the MIT license; files derived from ETP retain its Apache 2.0 license.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: No crowdsourcing or experiments with human subjects are reported.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human participants are involved in the reported experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Section 4 specifies the evaluated LLMs and calls; Appendix J describes the separate model-generated prose study. The reproducibility disclosure describes manuscript assistance.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.